Normalização e Desnormalização

Prof. Me. ***Orlando*** *da Silva Junior*

O que fizemos até agora?

- ✓ O **esquema de banco de dados relacional** consiste em uma série de esquemas de relação;
- ✓ Cada **esquema de relação** consiste em uma série de atributos;
- ✓ Os **atributos** são agrupados para formar um esquema de relação;

- Até o momento, projetamos os esquemas de relação utilizando:
  - ✓ Bom senso;
  - ✓ Modelo de dados conceitual (MER).

- Será que existe um modo formal de análise para agrupar os atributos?

Avaliação de **qualidade**

- Podemos avaliar a **qualidade** dos esquemas relacionais em dois níveis:
  - **Nível lógico:** como os usuários interpretam os esquema de relação e o significado de seus atributos;
  - **Nível físico:** como as tuplas em uma relação da base são armazenadas e atualizadas.



Projeto de BD

- O projeto de BD pode ser realizado por meio de duas técnicas:
  - **Bottom-up (projeto por síntese):** considera os relacionamentos básicos entre atributos individuais como ponto de partida e os utiliza para construir esquemas de relação;



  - **Top-down (projeto por análise):** agrupa os atributos em relações que existem naturalmente juntas (ex.: fatura, formulário, relatório).



Projeto de BD

- O projeto de BD relacional produto um conjunto de relações.
- Os objetivos implícitos da atividade de projeto são:
  - Preservação da informação;
  - Redundância mínima.

- **Preservar a informação** significa prover manutenção de todos os conceitos, incluindo os tipos de atributo, tipos de entidade e tipos de relacionamento.
  - Isto é necessário porque a informação é muito difícil de se quantificar!

- **Minimizar a redundância** implica em diminuir o armazenamento redundante da mesma informação e reduzir a necessidade de múltiplas atualizações para manter a consistência entre diversas cópias da mesma informação em resposta a eventos do mundo real que exijam fazer uma atualização.

Diretrizes

Vamos estabelecer algumas **diretrizes** antes de vermos a **teoria formal**?

- Como podemos garantir que a semântica dos atributos seja clara no esquema de relação?
- Como reduzir a informação redundante nas tuplas?
- Como reduzir os valores NULL nas tuplas?
- Como reprovar a possibilidade de gerar tuplas falsas?



Raciocinando...

- **Como podemos garantir que a semântica dos atributos seja clara no esquema de relação?**
  - Quanto mais fácil for explicar a semântica da relação, melhor será o projeto do esquema da relação.

- **Como reduzir a informação redundante nas tuplas?**
  - O armazenamento de junções naturais de relações da base leva a anomalias de atualização, que são incoerências indesejáveis nos dados.

- **Como reduzir os valores NULL nas tuplas?**
  - Agrupar muitos atributos em uma relação “gorda” que não são aplicáveis a todas as tuplas na relação gera muitos NULLs.

- **Como reprovar a possibilidade de gerar tuplas falsas?**
  - Tuplas falsas são informação não válida e podem ser evitadas com o uso correto dos operadores de projeção (π) e junção natural ($\star$).

Diretrizes informais

- **Diretriz 1**
  - ✓ Projete um esquema de modo que seja fácil explicar seu significado;
  - ✓ Não combine atributos de vários tipos de entidade e de relacionamento em uma única relação.



- **Diretriz 2**
  - ✓ Projete os esquemas de relação de modo que nenhuma anomalia de atualização esteja presente nas relações;
  - ✓ Se houver alguma anomalia, anote-as claramente e cuide para que os programas que atualizam o banco de dados operem corretamente.



Diretrizes informais

- **Diretriz 3**
  - ✓ Ao máximo possível, evite colocar atributos em uma relação da base cujos valores podem ser NULL com frequência;
  - ✓ Se os NULL forem inevitáveis, garanta que eles se apliquem apenas em casos excepcionais, e não à maioria das tuplas na relação;
  - ✓ Use o espaço de maneira eficaz e evite junções com valores NULL.



- **Diretriz 4**
  - ✓ Projete esquemas de relação de modo que possam ser unidos com condições de igualdade sobre atributos que são pares relacionados corretamente (chave primária, chave estrangeira) de um modo que garanta que nenhuma tupla falsa será gerada;
  - ✓Evite relações com atributos correspondentes que não sejam combinações, pois a junção sobre tais atributos pode produzir tuplas falsas.



Dependência funcional

- Uma **dependência funcional** é uma restrição entre dois conjuntos de atributos do banco de dados:
  - $X \rightarrow Y$ é uma restrição entre os subconjuntos X e Y de R especificada sobre possíveis tuplas que podem formar um estado de relação r de R.
  - A restrição é: quaisquer tuplas t_1 e t_2 em r que tenham $t_1[X] = t_2[X]$ devem ter $t_1[Y] = t_2[Y]$
  - "X determina funcionalmente Y" ou "Y depende funcionalmente de X"

- A dependência funcional é uma propriedade da semântica dos atributos.
  - Como projetistas, utilizamos o nosso conhecimento dos atributos para especificar as dependências funcionais que devem ser mantidas em todos os estados da relação.

Qual está correto?

CPF	Nome
160.984.172-70	João
676.124.010-70	Maria
652.820.343-10	Pedro
304.636.664-70	Ana
375.437.684-53	Paulo

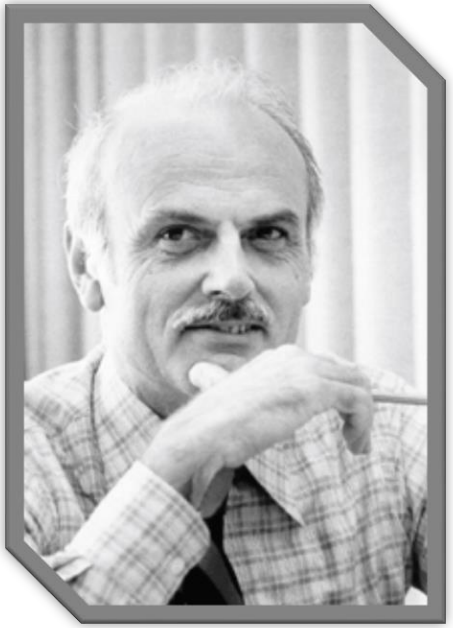
CPF	Nome
160.984.172-70	João
676.124.010-70	Maria
652.820.343-10	Pedro
304.636.664-70	Ana
304.636.664-70	Paulo

CPF	Nome
160.984.172-70	João
676.124.010-70	Maria
652.820.343-10	Pedro
304.636.664-70	Ana
375.437.684-53	Paulo
443.118.308-66	Paulo

- Para saber qual está correto, precisamos consultar os **domínios** dos atributos:
  - Se o CPF precisa ser único, o segundo caso está incorreto!
  - Se o nome não precisa ser único, o primeiro caso está correto!

- Se, com o CPF, você sabe quem é a pessoa, podemos dizer que o CPF determina o nome da pessoa ($\mathrm{CPF} \rightarrow \mathrm{Nome}$)
  - O terceiro caso está correto apenas se entendermos que é possível existir mais de uma pessoa com o mesmo nome.

Normalização



- O **processo de normalização** foi proposto por Edgar F. Codd em 1972 e consiste em uma série de testes para certificar se um esquema de relação satisfaz certa **forma normal**.

- O processo avalia cada relação em comparação com os critérios para as formas normais e decompõe as relações conforme a necessidade.

- Codd propôs três formas normais e todas elas estão baseadas nas dependências funcionais entre os atributos de uma relação.

- A 4ª e a 5ª formas normais foram propostas posteriormente com base nos conceitos de dependências multivaloradas e de junção, respectivamente.

Normalização **na prática**

- A **forma normal** indica o grau ao qual a relação foi normalizada, que é forma normal mais alta a que ela atende.
  - De maneira prática, vamos apenas até à 3ª Forma Normal;
  - Em algumas situações (desempenho!), podemos ir até à 2ª Forma Normal.

- Apesar de a 2ª e a 3ª formas atacarem problemas diferentes da 1ª forma, é comum segui-los na sequência.
  - Por definição, uma relação na 2ª Forma Normal já satisfaz a 1ª Forma Normal.

- Alguns chegam a citar +10 FNs diferentes!

Primeira forma normal (**1FN**)

- A 1FN afirma que o domínio de um atributo deve incluir apenas valores atômicos e que o valor de qualquer atributo em uma tupla deve ser um único valor do domínio desse atributo

- A 1FN reprova:
  - Ter um conjunto de valores;
  - Uma tupla de valores;
  - Combinação de conjuntos e tuplas de valores;
  - Relações dentro de relações;
  - Relações como valores de atributo dentro de tuplas.

(a)

DEPARTAMENTO



(b)

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente	Dlocal
Pesquisa	5	33344555587	Santo André, Itu, São Paulo
Administração	4	98765432168	Mauá
Matriz	1	88866555576	São Paulo

(c)

DEPARTAMENTO

Dnome	Dnumero	Cpf_gerente	Dlocal
Pesquisa	5	33344555587	Santo André
Pesquisa	5	33344555587	Itu
Pesquisa	5	33344555587	São Paulo
Administração	4	98765432168	Mauá
Matriz	1	88866555576	São Paulo

Processo de normalização na 1FN.
(ELMASRI & NAVATHE, 2011)

Segunda forma normal (**2FN**)

- Na 2FN, todos os atributos que não fazem parte da chave primária são dependentes dela.

BOLETIM (matricula-aluno, codigo-materia, numero-prova, nota, data-da prova, nome-aluno, endereço-aluno, nome-matéria)

Fazendo a análise da dependência funcional...

- matricula-aluno, codigo-materia, numero-prova → nota
- codigo-materia, numero-prova → data-da-prova
- matricula-aluno → nome-aluno, endereço-aluno
- codigo-matéria → nome-materia

Normalizando, teremos:

BOLETIM (matricula-aluno, codigo-materia, numero-prova, nota)

PROVA (codigo-materia, numero-prova, data-da-prova)

ALUNO (matricula-aluno, nome-aluno, endereço-aluno)

MATERIA (codigo-materia, nome-materia)

Terceira forma normal (**3FN**)

- Na 3FN, todos os atributos devem ser mutuamente independentes.

```
PEDIDO = {numero-pedido, codigo-cliente, data-pedido, nome-cliente,
          codigo-cidade-cliente, nome-cidade-cliente}
```

Fazendo a análise da dependência funcional...

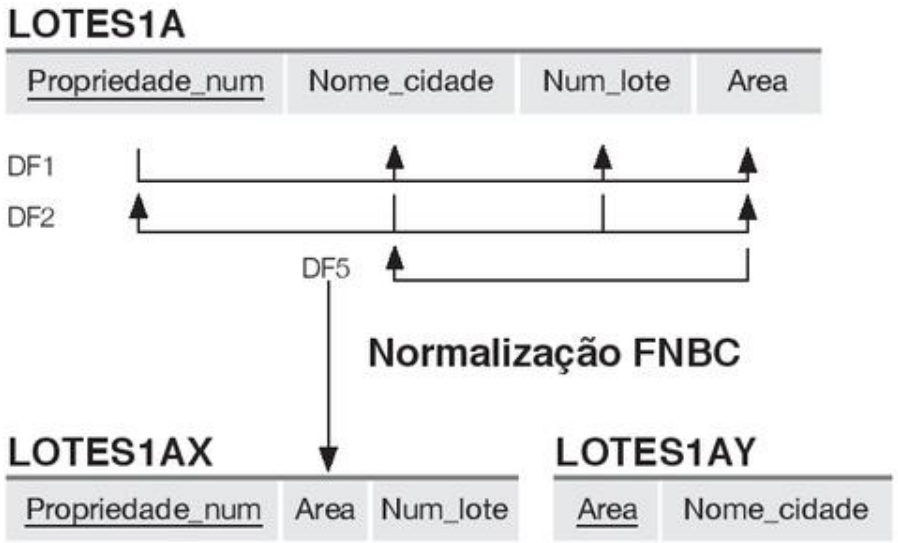
- matricula-aluno, codigo-materia, numero-prova → nota
- numero-pedido → codigo-cliente
- numero-pedido → data-pedido
- codigo-cliente → nome-cliente
- codigo-cliente → codigo-cidade-cliente
- codigo-cidade-cliente → nome-cidade-cliente

Ou seja...

- numero-pedido → codigo-cliente → nome-cliente
- numero-pedido → codigo-cliente → codigo-cidade-cliente
- numero-pedido → codigo-cliente → codigo-cidade-cliente → nome-cidade-cliente

Forma normal de **Boyce-Codd** (**FNBC**)

- A FNBC foi proposta como uma forma mais simples da 3FN, mas descobriu-se que ela era mais rigorosa.
  - Cada relação em FNBC está na 3FN;
  - Uma relação na 3FN não necessariamente está na FNBC.
- Na FNBC, não há dependências entre atributos não-chave.



Normalização FNBC.
(ELMASRI & NAVATHE, 2011)

ENSINA

Aluno	Disciplina	Professor
Lima	Banco de dados	Marcos
Silva	Banco de dados	Navathe
Silva	Sistemas operacionais	Omar
Silva	Teoria	Charles
Souza	Banco de dados	Marcos
Souza	Sistemas operacionais	Antonio
Wong	Banco de dados	Gomes
Zelaya	Banco de dados	Navathe
Lima	Sistemas operacionais	Omar

Exemplo de relação que está na 3FN, mas não na FNBC.
(ELMASRI & NAVATHE, 2011)

Desnormalização

- O objetivo final da normalização é separar atributos em tabelas e minimizar a redundância, evitando anomalias de atualização que levam a um *overhead* de processamento extra para manter a consistência no BD.

- As formas normais podem ser sacrificadas em favor de uma execução mais rápida de consultas e transações que ocorrem com frequência.

- Quando armazenamos o projeto lógico em uma FN mais fraca (FN3 para FN2, *e.g.*), estamos **desnormalizando** as relações.

Desnormalização

- A **desnormalização** é o processo de armazenar a junção de relações na forma normal mais alta como uma relação da base, que está em uma forma normal mais baixa.
  - A desnormalização também é uma decisão de projeto!

- Em geral, incluímos os atributos de uma tabela S em outra tabela R.
  - Os atributos de S são necessários com frequência;
  - Evitamos juntar R e S para otimizar consultas ou produzir relatórios;
  - Criamos redundância!

- **Existe um *trade-off* entre:** a atualização adicional necessária para manter a consistência dos atributos redundantes e o esforço necessário para realizar uma junção para incorporar atributos adicionais necessários no resultado.

Desnormalização / **Exemplo**

A seguinte relação corresponde ao relatório “Lista de Tarefas do Funcionário”

TAREFA (Func_id, Proj_id, Nome_func, Cargo_func,
Porc_atribuida, Nome_proj, Id_ger_proj, Nome_ger_proj)

Ela está na 1FN, porque:

- Proj_id → Nome_proj, Id_ger_proj
- Id_ger_proj → Nome_ger_proj
- Func_id → Nome_func, Cargo_func

Mas poderia estar na 2FN (ou 3FN) com as seguintes relações:

FUNC (Func_id, Nome_func, Cargo_func)
PROJ (Proj_id, Nome_proj, Id_ger_proj)
FUNC_PROJ (Func_id, Proj_id, Porc_atribuída)

Assim, o relatório seria gerado pela consulta:

$$((\text{FUNC_PROJ} \star \text{FUNC}) \star \text{PROJ}) \bowtie_{\text{PROJ.Id_ger_proj} = \text{FUNC.Id_func}} \text{FUNC}$$

Vamos **validar** o que estudamos até agora?

1. Reúna-se com até 2 colegas e escreva o algoritmo (passo-a-passo) para aplicar as três formas normais.



2. Ainda com seus colegas, descreva um caso já vivenciado por vocês em que foi preciso desnormalizar o BD. Cite o contexto e as etapas.